

دليل التشغيل المفصل

دليل بناء وتشغيل محاكي جدولة وحدة المعالجة المركزية

CPU Scheduling Simulator Using Appropriate Data Structures

إعداد	مؤمن عبدالله الشامي
إشراف	د. بلال مرشد
لغة التنفيذ	C++17
العام الجامعي	2026 م

تنفيذ الخوارزميات باستخدام Array و Queue و Priority Queue و Linked List و Stack

1. الغرض من الدليل

يوضح هذا الدليل متطلبات بناء وتشغيل مشروع محاكي جدولة وحدة المعالجة المركزية على نظام Windows باستخدام Dev-C++ و Code::Blocks و Visual Studio Code ، إضافة إلى طريقة التجميع المباشر بواسطة g++ . كما يوضح طريقة الحصول على المشروع من GitHub ، والتحقق من النتائج، ومعالجة أكثر أخطاء البناء والتشغيل شيوعًا.

البرنامج غير تفاعلي؛ إذ توجد العمليات وقيمة Time Quantum داخل الكود. وبعد نجاح التجميع يبدأ التنفيذ مباشرة، ويعرض نتائج FCFS و SJF و Priority Scheduling و Round Robin من دون طلب إدخال أي قيم من لوحة المفاتيح.

2. ملفات المشروع

تتكون حزمة التسليم من الملفات والمجلدات الآتية:

المسار	المحتوى
src/ CPU_Scheduling_Simulator.cpp	الكود المصدري الرئيس
docs/Project_Report_AR.pdf	التقرير الرسمي للمشروع
docs/Run_Guide_AR.pdf	دليل التشغيل
docs/Code_Explanation_AR.pdf	شرح الكود
output/Sample_Output.txt	نسخة نصية من نتيجة تشغيل صحيحة
README.md	تعريف مختصر بالمشروع وأوامر البناء
.gitignore	استبعاد ملفات البناء المؤقتة من GitHub

3. متطلبات التشغيل

يلزم توفر أحد الخيارات الآتية:

- Dev-C++ مزود بـ مترجم MinGW يدعم C++17.
 - Code::Blocks مزود بـ مترجم MinGW/GCC.
 - Visual Studio Code مع إضافة C/C++ و مترجم g++ مثبت.
 - مترجم g++ مستقل يمكن استدعاؤه من Command Prompt أو PowerShell.
- ولا يحتاج المشروع إلى مكتبات خارجية؛ لأنه يستخدم مكتبات C++ القياسية فقط.

4. الحصول على المشروع

4.1 من الحزمة المضغوطة

1. يُفك ضغط ملف المشروع في مجلد قصير وواضح، مثل:

```
C:\cpu-scheduling-simulation-ds
```

1. يُتأكد من وجود الملف:

```
C:\cpu-scheduling-simulation-ds\src\CPU_Scheduling_Simulator.cpp
```

1. لا يُشغل ملف C++ بالنقر المزدوج؛ بل يُفتح داخل بيئة التطوير أو يُجمع باستخدام المترجم.

4.2 Download ZIP بواسطة GitHub

1. يُفتح رابط المستودع:

```
https://github.com/alimohammedabotaleb-hub/cpu-scheduling-simulation-ds
```

1. يُختار الزر **Code** ثم **Download ZIP**.

2. يُفك ضغط الملف بعد اكتمال التنزيل.

3. يُفتح المجلد الناتج، ثم يُتبع أحد مسارات التشغيل الواردة في الأقسام التالية.

4.3 GitHub بواسطة Git

بعد تثبيت Git ، يُفتح Command Prompt في المجلد المراد حفظ المشروع داخله، ثم ينفذ:

```
git clone https://github.com/alimohammedabotaleb-hub/cpu-scheduling-simulation-ds
cd cpu-scheduling-simulation-ds
```

للتحقق من الملفات:

```
dir
dir src
```

5. التشغيل باستخدام Dev-C++

5.1 فتح الملف مباشرة

1. يُشغل Dev-C++.

2. من القائمة العلوية يُختار **File** ثم **Open**.

3. يُحدد الملف:

```
src\CPU_Scheduling_Simulator.cpp
```

1. من قائمة **Execute** يُختار **Compile & Run**، أو يُضغط المفتاح **F11** بحسب إصدار البرنامج.

2. عند نجاح التجميع تظهر نافذة Console وتبدأ المحاكاة تلقائيًا.

5.2 إنشاء مشروع Console عند تعذر الفتح المباشر

1. يُختار **File** ثم **New** ثم **Project**.

2. يُحدد **Console Application**.

3. تُحدد لغة **C++**.

4. يُكتب اسم المشروع، مثل:

```
CPU_Scheduling_Simulator
```

1. بعد إنشاء المشروع، يُحذف محتوى ملف **main.cpp** الافتراضي أو يُزال الملف من المشروع.

2. يُختار **Project** ثم **Add to Project**، ويضاف ملف:

```
CPU_Scheduling_Simulator.cpp
```

1. يُنفذ **Compile & Run**.

5.3 ضبط معيار C++17 في Dev-C++

إذا ظهر خطأ متعلق بميزات اللغة، يُضبط معيار C++17 كما يأتي:

1. يُفتح **Tools** ثم **Compiler Options**.

2. يُحدد مربع إضافة أوامر للمترجم.

3. يُضاف الخيار:

```
-std=c++17
```

1. يُعاد تنفيذ **Compile & Run**.

6. التشغيل باستخدام Code::Blocks

6.1 إنشاء مشروع جديد

1. يُشغل **Code::Blocks**.

2. يُختار **File** ثم **New** ثم **Project**.

3. يُحدد **Console application** ثم **Go**.

4. تُحدد لغة **C++**.

5. يُكتب اسم المشروع ويُحدد مكان حفظه.

6. بعد إنشاء المشروع، يظهر ملف **main.cpp** افتراضي.

7. يُزال **main.cpp** من المشروع لتجنب وجود دالتين باسم **main**.

8. يُنقر بزر الفأرة الأيمن على اسم المشروع، ثم **Add files**.

9. يضاف الملف:

```
src\CPU_Scheduling_Simulator.cpp
```

1. يُختار **Build and Run** أو يُضغط **F9**.

6.2 ضبط معيار C++17 في Code::Blocks

1. يُفتح **Project** ثم **Build options**.

2. يُحدد اسم المشروع من الجانب الأيسر.

3. من **Compiler settings** ثم **Compiler flags** يُفعل الخيار المتعلق بمعيار C++17 ، أو يضاف في **Other compiler options**:

```
-std=c++17
```

1. يُنفذ **Rebuild** ثم **Run**.

7. التشغيل باستخدام Visual Studio Code

7.1 المتطلبات الأولية

يلزم توفر ما يأتي:

1. برنامج **Visual Studio Code**.

2. إضافة **C/C++** المنشورة بواسطة **Microsoft**.

3. مترجم **MinGW-w64** أو **MSYS2** يتضمن **g++.exe**.

4. إضافة مسار مجلد `bin` الخاص بالمترجم إلى متغير البيئة `Path` في Windows. للتحقق من المترجم، يُفتح Terminal داخل VS Code وينفذ:

```
g++ --version
```

ظهور رقم إصدار المترجم يعني أن الإعدادات صحيحة.

7.2 فتح المشروع

1. يُختار `File` ثم `Open Folder`.
2. يُحدد مجلد `cpu-scheduling-simulation-ds` كاملاً.
3. يُفتح الملف `src/CPU_Scheduling_Simulator.cpp`.
4. يُفتح الطرفية من `Terminal` ثم `New Terminal`.

7.3 التجميع والتشغيل من طرفية VS Code

من جذر المشروع ينفذ أمر التجميع في PowerShell أو Command Prompt:

```
g++ -std=c++17 -Wall -Wextra -pedantic src/CPU_Scheduling_Simulator.cpp -o  
CPU_Scheduling_Simulator.exe
```

ثم ينفذ البرنامج في PowerShell:

```
.\CPU_Scheduling_Simulator.exe
```

أو في Command Prompt:

```
CPU_Scheduling_Simulator.exe
```

7.4 التشغيل بزر Run

يمكن استخدام `Run` ثم `Run Without Debugging` بعد اختيار (GDB/LLDB) `C++` وإنشاء إعداد البناء المقترح من VS Code. ومع ذلك، يظل التجميع المباشر من الطرفية أكثر وضوحًا عند التحقق الأول من المشروع.

8. التشغيل المباشر باستخدام g++ على Windows

8.1 فتح موجه الأوامر في مجلد المشروع

يمكن فتح مجلد المشروع في File Explorer ، ثم كتابة `cmd` في شريط العنوان والضغط على Enter. بعد ذلك يظهر موجه الأوامر داخل المجلد مباشرة.

8.2 التحقق من توفر المترجم

```
g++ --version
```

8.3 بناء الملف التنفيذي

```
g++ -std=c++17 -Wall -Wextra -pedantic src\CPU_Scheduling_Simulator.cpp -o  
CPU_Scheduling_Simulator.exe
```

إذا لم تظهر رسائل خطأ، يكون الملف التنفيذي قد أنشئ في جذر المشروع.

8.4 تشغيل البرنامج

```
CPU_Scheduling_Simulator.exe
```

8.5 حفظ نتيجة التشغيل في ملف

```
CPU_Scheduling_Simulator.exe > output\Sample_Output.txt
```

يعيد هذا الأمر كتابة ملف النتائج. ولإضافة نتيجة جديدة دون حذف السابقة يستخدم:

```
CPU_Scheduling_Simulator.exe >> output\Sample_Output.txt
```

9. التشغيل باستخدام g++ على Linux

من جذر المشروع:

```
g++ -std=c++17 -Wall -Wextra -pedantic src/CPU_Scheduling_Simulator.cpp -o cpu_scheduler
./cpu_scheduler
```

ولحفظ النتيجة:

```
./cpu_scheduler > output/Sample_Output.txt
```

10. ما يحدث عند التشغيل

لا تظهر قائمة إدخال ولا يطلب البرنامج كتابة عدد العمليات. يبدأ المسار الآتي تلقائيًا:

1. تحميل P1 و P2 و P3 و P4 و P5 من العينة المعرفة داخل الكود.
 2. اعتماد Time Quantum المدمجة لخوارزمية Round Robin.
 3. عرض جدول العمليات الأساسية.
 4. تنفيذ FCFS وعرض نتائجها.
 5. تنفيذ SJF وعرض نتائجها.
 6. تنفيذ Priority Scheduling وعرض نتائجها.
 7. تنفيذ Round Robin وعرض نتائجها.
 8. عرض جدول مقارنة الأداء.
- ويشمل مخرج كل خوارزمية:

- ترتيب التنفيذ.
- مخطط Gantt النصي.
- CT و WT و TAT لكل عملية.
- Average Waiting Time.
- Average Turnaround Time.
- CPU Idle Time. و CPU Busy Time.
- CPU Utilization.
- سجل التنفيذ العكسي الناتج من Stack.

11. التحقق من نجاح التشغيل

يُعد التشغيل ناجحًا عند تحقق العلامات الآتية:

- ظهور عنوان البرنامج وجدول العمليات المدمجة.
 - ظهور الأقسام الأربعة FCFS و SJF و Priority و Round Robin.
 - ظهور جدول لكل خوارزمية من دون قيم سالبة في WT أو TAT.
 - ظهور نسبة CPU Utilization ضمن المجال من 0% إلى 100%.
 - ظهور قسم Performance Comparison في نهاية التنفيذ.
 - عودة البرنامج إلى النظام دون رسالة خطأ.
- يمكن مقارنة المخرجات الظاهرة بمحتوى الملف:

```
output\Sample_Output.txt
```

12. تعديل العينة المدمجة عند الحاجة

لا يلزم أي تعديل لتشغيل نسخة التسليم. وإذا تطلب الاختبار الفني تغيير العمليات، فإن التعديل يقتصر على المصفوفة المعرفة في الجزء المسمى **Built-in Sample Data** داخل ملف المصدر. يمثل كل سجل القيم الآتية بالترتيب:

```
{"P1", arrivalTime, burstTime, priority}
```

ويجب الالتزام بالقواعد الآتية:

- يكون Process ID فريدًا.
 - يكون Arrival Time صفرًا أو أكبر.
 - يكون Burst Time أكبر من صفر.
 - يعني رقم Priority الأصغر أولوية أعلى.
 - تُعدل قيمة المتغير المحلي الثابت **timeQuantum** داخل الدالة **main** عند الحاجة مع إبقائها أكبر من صفر.
- بعد أي تعديل يجب إعادة التجميع؛ لأن تشغيل الملف التنفيذي القديم لا يعكس تغييرات ملف المصدر.

13. استكشاف الأخطاء وإصلاحها

13.1 الخطأ: g++ is not recognized

السبب: المترجم غير مثبت أو أن مساره غير مضاف إلى **Path**.
المعالجة:

1. تثبيت MinGW-w64 أو MSYS2.
2. تحديد مجلد **bin** الذي يحتوي على **g++.exe**.
3. إضافته إلى متغير البيئة **Path**.
4. إغلاق نافذة Terminal وفتحها من جديد.
5. تنفيذ **g++ --version** للتحقق.

13.2 الخطأ: No such file or directory

السبب: الأمر ينفذ من مجلد مختلف أو أن اسم الملف/المسار غير صحيح.
المعالجة:

- فتح الطرفية في جذر المشروع.
- تنفيذ `dir src` في Windows أو `ls src` في Linux.
- نسخ الاسم كما هو، مع الانتباه إلى الامتداد `..cpp`.

13.3 الخطأ: multiple definition of main

السبب: وجود ملف `main.cpp` افتراضي مع ملف المشروع، وكلاهما يحتوي على دالة `main`.
 المعالجة: إزالة ملف `main.cpp` الافتراضي من المشروع، والإبقاء على `CPU_Scheduling_Simulator.cpp` فقط ضمن ملفات البناء.

13.4 الخطأ: ميزات C++ غير معروفة

السبب: استخدام معيار قديم للغة أو مترجم قديم.
 المعالجة: إضافة الخيار:

```
-std=c++17
```

وفي حال استمرار الخطأ يلزم تحديث المترجم.

13.5 النافذة السوداء تُغلق بسرعة

السبب: تشغيل الملف التنفيذي بالنقر المزدوج يؤدي إلى إغلاق النافذة بعد انتهاء البرنامج.
 المعالجة: تشغيل البرنامج من داخل Dev-C++ أو Code::Blocks أو من Command Prompt/PowerShell. ويمكن في Command Prompt استخدام:

```
CPU_Scheduling_Simulator.exe  
pause
```

13.6 لا تظهر التعديلات الجديدة

السبب: تشغيل ملف تنفيذي قديم بعد تعديل المصدر.
 المعالجة: تنفيذ `Rebuild` أو حذف الملف التنفيذي القديم ثم إعادة التجميع.

13.7 ظهور رموز غير مفهومة في النص العربي

المخرجات البرمجية الأساسية باللغة الإنجليزية لتجنب اختلاف ترميز الطرفيات. أما ملفات التوثيق العربية فتفتح باستخدام Word أو قارئ PDF يدعم Unicode. إذا احتوى المصدر على تعليق عربي، فيُحفظ بترميز UTF-8.

13.8 Permission denied عند إنشاء الملف التنفيذي

السبب: الملف التنفيذي يعمل حاليًا أو أن المجلد محمي.
 المعالجة: إغلاق نافذة البرنامج، وإيقاف أي نسخة قيد التشغيل، ثم البناء في مجلد عادي مثل Documents أو Desktop بدل مجلدات النظام المحمية.

13.9 ظهور قيم CPU Utilization متساوية

لا يمثل ذلك خطأ بالضرورة. عند استخدام مجموعة العمليات نفسها وإهمال كلفة التبديل السياقي، تبقى مدة انشغال المعالج متساوية لمجموع Burst Time في جميع الخوارزميات. لذلك قد تتساوى نسبة الاستخدام، في حين تختلف أزمنة الانتظار والدوران وترتيبات التنفيذ.

13.10 ظهور قيم انتظار سالبة

يشير ذلك إلى خطأ في الحساب أو إلى بدء عملية قبل Arrival Time. يجب التحقق من أن:

```
TAT = CT - AT
WT = TAT - BT
```

وأن الجدول لا يضيف إلى قائمة الجاهزين إلا العمليات التي تحقق $Arrival\ Time \leq Current\ Time$.

14. رفع المشروع إلى GitHub

14.1 إنشاء مستودع جديد

1. يُسجل الدخول إلى GitHub.
2. يُختار `New repository`.
3. يعتمد اسم واضح، مثل:

```
cpu-scheduling-simulation-ds
```

1. يضاف وصف مختصر:

```
C++ simulation of FCFS, SJF, Priority Scheduling, and Round Robin using appropriate data structures.
```

1. يحدد مستوى الظهور المطلوب وفق تعليمات المقرر: `Public` أو `Private`.
2. يُنشأ المستودع من دون إضافة ملفات تلقائية عند وجود README محلي جاهز.

14.2 رفع الملفات باستخدام Git

من جذر المشروع تنفذ الأوامر الآتية بعد استبدال الرابط بعنوان المستودع الفعلي:

```
git init
git add .
git commit -m "Initial CPU scheduling simulator submission"
git branch -M main
git remote add origin https://github.com/alimohammedabotaleb-hub/cpu-scheduling-simulation-ds.git
git push -u origin main
```

إذا كان المستودع قد جُلب مسبقًا بواسطة `git clone`، يكفي بعد التعديلات:

```
git add .
git commit -m "Update project documentation and results"
git push
```

14.3 رفع الملفات من واجهة GitHub

يمكن استخدام `Add file` ثم `Upload files`، وسحب مجلدات `src` و `docs` و `output` مع `README.md` و `.gitignore`، ثم كتابة وصف للتحديث واختيار `Commit changes`.

14.4 التحقق من المستودع

يُتحقق من الأمور الآتية بعد الرفع:

- ظهور ملف المصدر داخل `src`.
- ظهور ملفات التقرير والدليل والشرح داخل `docs`.
- ظهور README تلقائيًا في الصفحة الرئيسية.
- عمل رابط فتح ملف المصدر.

- عدم رفع ملفات تنفيذية أو مجلدات بناء غير ضرورية.
- نسخ رابط المستودع الصحيح وإدراجه في غلاف التقرير.

15. أوامر سريعة

Windows

```
g++ -std=c++17 -Wall -Wextra -pedantic src\CPU_Scheduling_Simulator.cpp -o
CPU_Scheduling_Simulator.exe
CPU_Scheduling_Simulator.exe
CPU_Scheduling_Simulator.exe > output\Sample_Output.txt
```

Linux

```
g++ -std=c++17 -Wall -Wextra -pedantic src/CPU_Scheduling_Simulator.cpp -o cpu_scheduler
./cpu_scheduler
./cpu_scheduler > output/Sample_Output.txt
```

16. قائمة التحقق النهائية

الحالة	بند التحقق
☐	ملف المصدر موجود ويُفتح بصورة صحيحة
☐	البناء بمعيار C++17 ناجح
☐	التشغيل يتم من دون إدخال يدوي
☐	الخوارزميات الأربع تظهر في المخرجات
☐	WT و TAT والمتوسطات تظهر لكل خوارزمية
☐	CPU Utilization تظهر لكل خوارزمية
☐	مخطط Gantt يظهر بصورة صحيحة
☐	ملف Sample_Output يطابق التشغيل
☐	ملفات التقرير والشرح والدليل موجودة
☐	مستودع GitHub يحتوي ملفات المشروع